

## Construction of Top-Down and Bottom-up parsers.

→ Consider a calculator application that accepts arithmetic expression as  $a + b * c$ . The compiler needs to check whether the expression follows the grammar, identify parsing conflicts, recover from errors, and generate a syntax tree.

The CFG used is:

$$E \rightarrow E + T / T$$

$$T \rightarrow T * F / F$$

$$F \rightarrow (E) / id$$

### Top down parsing:

The parser starts from the symbol  $E$  and tries to generate the given input.

→ The parser generates the structure where  $b * c$  is treated as a term and added to  $a$ .



However, the grammar containing left recursion such as  $E \rightarrow E + T$ . This creates a conflict for a recursive - descent parser.

→ The grammar can be rewritten as :

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' / \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' / \epsilon$$

$$F \rightarrow (E) / id$$

Bottom - up parsing :

the parser starts with the input symbol and gradually reduces them according to the grammar until it reaches the start symbol  $E$ .

→ A shift - reduce parser is used for bottom-up parsing. It shifts input symbols onto the stack and reduces them when a valid grammar production is identified.



## Error Recovery :

If the user enters an incorrect expression

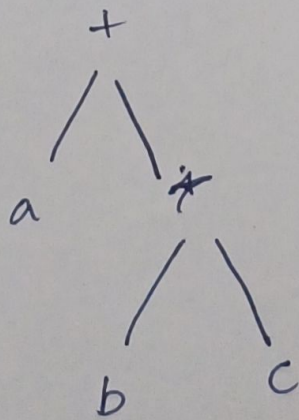
such as :

$a + * b$

The parser identifies an error because an operand is expected after the + operators, but \* is found.

→ This allow the compiler to report the error without completely stopping the parsing process.

Syntax tree :



Shows that multiplication is performed before addition.